HookRelay

A source-agnostic webhook relay built for reliable event delivery.

Receive, verify, store, and forward webhook events to registered endpoints with automatic retries, payload signing, and delivery logs.

Built by Reabot6 at **SFG** · Open source under the MIT License

---

Overview

Webhook integrations can become difficult to manage when multiple external services need to communicate with different parts of an application. Each integration may have its own signature format, delivery behavior, and failure modes.

HookRelay provides a centralized layer for receiving and forwarding webhook events. It verifies supported source signatures, persists incoming events, and processes outbound deliveries through a background worker.

When a destination is temporarily unavailable, HookRelay retries delivery with exponential backoff and jitter. Each attempt can be tracked, and events that exhaust their retries can be replayed manually.

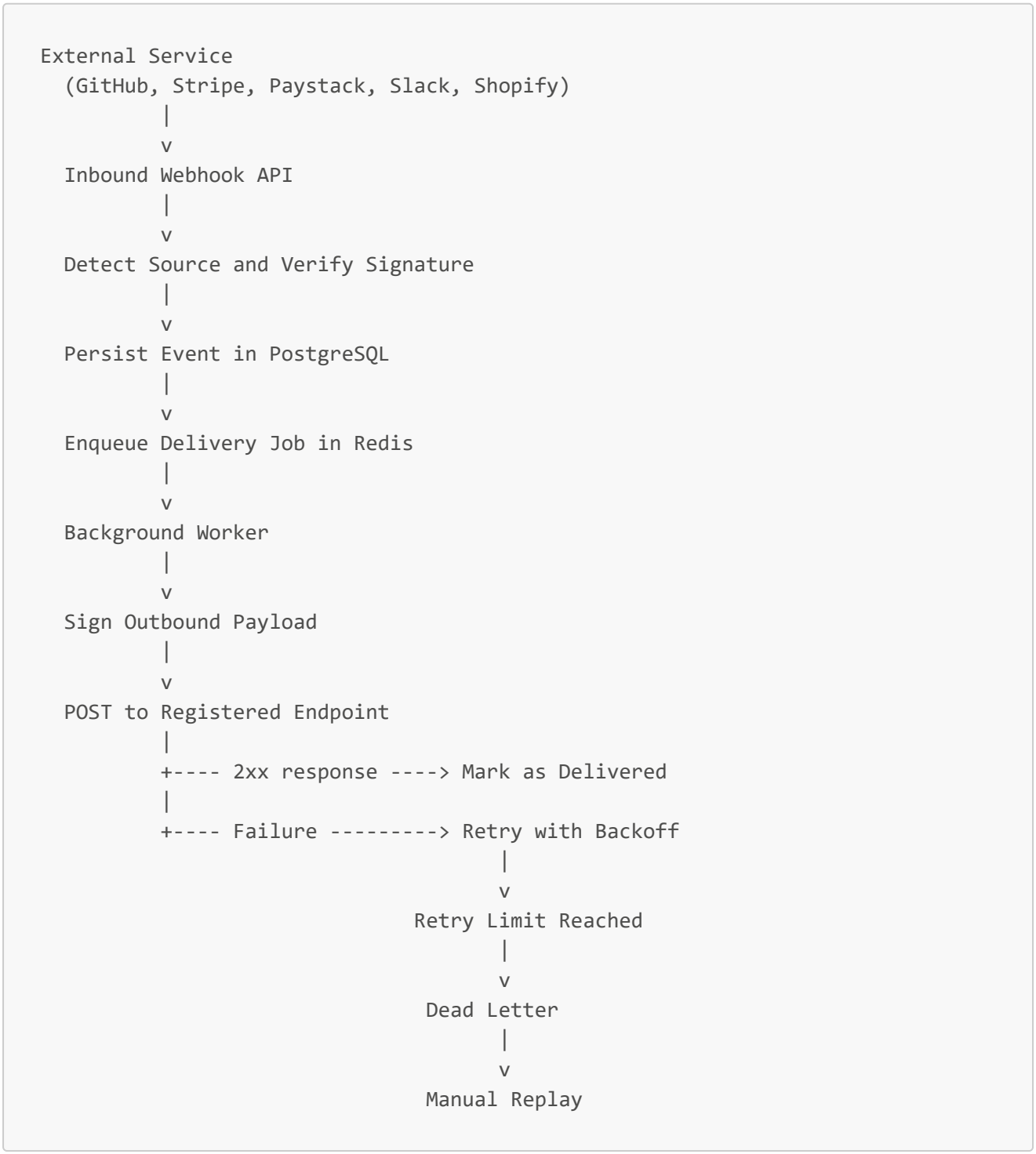
The goal is to make webhook delivery easier to manage without requiring every application to implement its own queue, retry mechanism, and delivery logging system.

Features

- **Centralized webhook ingestion** — receive events from supported external services through a unified API.
- **Source signature verification** — validate incoming webhook signatures using source-specific algorithms.
- **Persistent event storage** — store incoming events in PostgreSQL.
- **Asynchronous delivery** — process outbound webhook jobs using Redis and BullMQ.
- **Automatic retries** — retry failed deliveries with exponential backoff and jitter.
- **Payload signing** — sign outbound requests so receiving applications can verify their authenticity.
- **Delivery logs** — track delivery attempts and inspect recent activity.
- **Dead-letter handling** — retain deliveries that exhaust their configured retries for later investigation or replay.
- **Endpoint management** — register, update, list, and remove destination endpoints.
- **API authentication** — use JWT authentication for dashboard operations and API keys for API access.
- **Encrypted endpoint secrets** — protect stored endpoint secrets using AES-256-GCM.
- **Event simulation** — use the simulator to test supported event flows.

How It Works

HookRelay separates inbound event processing from outbound delivery.



Processing Flow

1. An external service sends a webhook to HookRelay's inbound API.
2. HookRelay identifies the source from request headers and verifies the signature when supported and configured.
3. The event is persisted in PostgreSQL.
4. A delivery job is queued in Redis through BullMQ.
5. A separate worker signs the outbound payload and sends it to the registered destination.
6. Successful deliveries are marked as delivered.
7. Failed deliveries are retried according to the configured schedule.

8. Deliveries that exhaust their retry attempts can be inspected and replayed.

Persisting events before queueing delivery helps preserve the event record if downstream processing fails. Actual guarantees depend on transaction handling, queue recovery, and the application's error-handling implementation.

Supported Sources

HookRelay supports source-specific signature verification for the following services.

Source	Signature Header	Algorithm	Replay Protection
GitHub	`X-Hub-Signature-256`	HMAC-SHA256, hexadecimal	Not implemented
Stripe	`Stripe-Signature`	HMAC-SHA256, hexadecimal	5-minute timestamp tolerance
Paystack	`X-Paystack-Signature`	HMAC-SHA512, hexadecimal	Not implemented
Slack	`X-Slack-Signature`	HMAC-SHA256, hexadecimal	5-minute timestamp tolerance
Shopify	`X-Shopify-Hmac-SHA256`	HMAC-SHA256, Base64	Not implemented

Replay protection is source-specific. A timestamp tolerance can limit the acceptance window for signed requests, but it does not automatically prevent every duplicate event.

Important: Signature verification must use the original request body in the format required by each provider. Verify the current implementation against the providers' requirements before using HookRelay with production webhooks.

Tech Stack

Layer	Technology
Runtime	Bun
Backend Framework	Express 5
Language	TypeScript, strict mode
Database	PostgreSQL
ORM	Drizzle ORM
Job Queue	Redis, BullMQ
Authentication	JWT, API keys
Secret Encryption	AES-256-GCM

Layer	Technology
Outbound Signatures	HMAC-SHA256
Infrastructure	Docker Compose

Project Structure

```
hookrelay/
└── src/
    ├── controller/
    │   ├── auth.ts
    │   ├── deliveries.ts
    │   ├── endpoint.ts
    │   ├── inbound.ts
    │   └── simulator.ts
    ├── db/
    │   ├── index.ts
    │   └── schema.ts
    ├── middleware/
    │   ├── has-api-key.ts
    │   └── is-auth.ts
    ├── queue/
    │   └── delivery.ts
    ├── routes/
    │   ├── auth.ts
    │   ├── deliveries.ts
    │   ├── endpoint.ts
    │   ├── inbound.ts
    │   └── simulator.ts
    ├── service/
    │   ├── encryption.ts
    │   └── verify-webhook.ts
    ├── shared/
    │   └── types.ts
    ├── validation/
    │   ├── auth.ts
    │   ├── endpoint.ts
    │   └── simulator.ts
    ├── index.ts
    └── worker.ts
```

Core Components

- **Controllers** handle authentication, endpoint management, inbound webhooks, deliveries, and event simulation.
- **Database layer** manages PostgreSQL connectivity and the Drizzle schema.
- **Queue layer** schedules and processes asynchronous delivery jobs.
- **Middleware** handles API-key and JWT authentication.

- **Services** implement source verification and endpoint-secret encryption.
- **Validation** keeps request validation separate from controller logic.
- **API server** accepts inbound requests and exposes the REST API.
- **Worker** processes queued outbound deliveries independently of the API server.

Getting Started

Prerequisites

Install the following:

- Bun
- Docker
- Docker Compose

You will also need a PostgreSQL database and Redis instance. Docker Compose can provide these locally if the project's configuration defines both services.

1. Clone the Repository

```
git clone https://github.com/reabot6/hookrelay.git
cd hookrelay
```

2. Install Dependencies

```
bun install
```

3. Start the Dependencies

```
docker compose up -d
```

Confirm that PostgreSQL and Redis are running before starting the application.

4. Configure Environment Variables

Create your local environment file:

```
cp .env.example .env
```

Configure the required values, including:

- `DATABASE_URL` — PostgreSQL connection string.

- `SECRET_KEY` — application secret used by the relevant authentication or cryptographic code.
- `ENCRYPT_KEY` — encryption key for endpoint secrets.
- Redis connection settings, if required by the project configuration.

Check `.env.example` and the source code for the complete list of required variables. Do not commit real credentials or secrets.

5. Generate an Encryption Key

Generate 32 random bytes and encode them as hexadecimal:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Use the generated value for `ENCRYPT_KEY` if the encryption service expects a 64-character hexadecimal key.

Keep this key secure. Losing it may make previously encrypted endpoint secrets impossible to decrypt.

6. Initialize the Database

```
bunx drizzle-kit push
```

This applies the Drizzle schema to the configured database. For production environments, use a controlled migration workflow rather than applying schema changes without review.

7. Start the API Server

In the first terminal:

```
bun dev
```

8. Start the Background Worker

In a second terminal, from the project root:

```
bun worker
```

The API server and worker are separate processes. Both must be running for queued events to be delivered.

> **Configuration note:** The commands above follow the project's documented workflow. Confirm the actual scripts in `package.json`, the required environment variables, and the Docker Compose service definitions before running the application.

API Reference

The API provides authentication, endpoint management, inbound webhook ingestion, delivery inspection, and replay operations.

The route prefixes below reflect the documented API surface. Exact URL prefixes and authentication requirements should be confirmed against the registered Express routes.

Authentication

Method	Route	Authentication	Description
`POST`	`/api/auth/register`	None	Create an account
`POST`	`/api/auth/login`	None	Authenticate and obtain a JWT
`PUT`	`/api/auth/api-key`	JWT	Generate an API key

Endpoint Management

The API supports endpoint creation, listing, retrieval, updates, and deletion.

Method	Route	Description
`POST`	`/endpoints`	Register an endpoint
`GET`	`/endpoints`	List endpoints
`GET`	`/endpoints/:id`	Retrieve an endpoint
`PUT`	`/endpoints/:id`	Update an endpoint
`DELETE`	`/endpoints/:id`	Delete an endpoint

The API-key and dashboard route prefixes may differ. Refer to the route definitions for the exact paths exposed by each authentication method.

Inbound Webhooks

Method	Route	Description
`POST`	`/api/inbound/:endpointId`	Receive an inbound webhook

External services send their webhook requests to the appropriate inbound URL.

The `endpointId` identifies the configured inbound destination or integration according to the application's data model.

Deliveries

Method	Route	Description
`GET`	`/deliveries/stats`	Retrieve delivery statistics

Method	Route	Description
GET	/deliveries/recent	Retrieve the latest 10 deliveries
GET	/deliveries/:endpointId	Retrieve delivery logs for an endpoint
POST	/deliveries/:callbackId/replay	Replay a failed or dead delivery

The exact route prefix and authorization requirements depend on the registered API and dashboard routes.

Outbound Webhook Format

HookRelay signs outbound payloads so receiving applications can verify that a request was signed using the configured endpoint secret.

Headers

Header	Description
X-HookRelay-Signature	HMAC-SHA256 signature of the payload
X-HookRelay-Event	Event type, such as payment.failed
X-HookRelay-Callback-Id	Unique identifier for the delivery

The signature uses the hrelay_sha256= prefix followed by the hexadecimal HMAC-SHA256 digest.

Verifying a Signature

The following Node.js example illustrates how a receiving application can verify a signature using the original request body.

```
const crypto = require("crypto");

function verifyHookRelaySignature(rawBody, signature, endpointSecret) {
  if (typeof signature !== "string") {
    return false;
  }

  const expected =
    "hrelay_sha256=" +
    crypto
      .createHmac("sha256", endpointSecret)
      .update(rawBody)
      .digest("hex");

  const expectedBuffer = Buffer.from(expected, "utf8");
  const receivedBuffer = Buffer.from(signature, "utf8");

  if (expectedBuffer.length !== receivedBuffer.length) {
    return false;
```

```
  }

  return crypto.timingSafeEqual(expectedBuffer, receivedBuffer);
}
```

Example usage:

```
const signature = req.headers["x-hookrelay-signature"];

if (
  !verifyHookRelaySignature(
    req.rawBody,
    signature,
    process.env.HOOKRELAY_ENDPOINT_SECRET
  )
) {
  return res.status(401).send("Invalid signature");
}
```

Important:

- `req.rawBody` must contain the exact bytes used to calculate the signature. A parsed and re-serialized JSON object may produce a different digest.
- The endpoint secret must match the secret configured for that destination.
- Use a constant-time comparison for signature verification.
- The receiving application should also validate the event schema and handle duplicate delivery attempts safely.

Adapt the raw-body handling to your web framework and confirm the signing format against HookRelay's implementation.

Retry Policy

HookRelay uses exponential backoff with jitter to avoid repeatedly overwhelming an unavailable destination.

The documented retry schedule is:

Attempt	Base Delay	Delay With Jitter
1	10 seconds	10–20 seconds
2	30 seconds	30–60 seconds
3	2 minutes	2–4 minutes
4	10 minutes	10–20 minutes
5	1 hour	1–2 hours

After five failed attempts, a delivery is marked as dead and can be replayed manually.

The configured timeout for each outbound request is 10 seconds.

Jitter randomizes retry timing to reduce the likelihood that multiple failed deliveries will retry simultaneously.

The precise meaning of an attempt number—initial delivery versus retry—should be confirmed against the BullMQ job configuration and worker implementation.

Security Considerations

Webhook infrastructure handles external input, credentials, and potentially sensitive event data. HookRelay includes several security-oriented components, but additional safeguards are important before production use.

- **Verify incoming signatures:** Validate each supported source using its official signing procedure.
- **Protect endpoint secrets:** Keep encryption keys outside version control and restrict access to them.
- **Verify outgoing signatures:** Receiving applications should validate `X-HookRelay-Signature` before processing events.
- **Protect API credentials:** Use secure storage for API keys and JWT signing secrets.
- **Validate requests:** Enforce payload-size limits, schema validation, and appropriate rate limits.
- **Prevent replay attacks:** Use timestamp validation and idempotent event processing where supported.
- **Restrict outbound destinations:** Validate destination URLs and block requests to private, loopback, and cloud metadata addresses where appropriate to mitigate server-side request forgery (SSRF).
- **Limit sensitive logging:** Avoid exposing secrets, authorization headers, or unnecessary personal data in delivery logs.
- **Secure production traffic:** Use HTTPS and appropriately restricted database and Redis access.

These measures should be reviewed against the current implementation rather than assumed to be fully enforced simply because the relevant components exist.

Changes From the Original Project

HookRelay was adapted from `conduit-engine` with a focus on consistent naming and clearer separation of responsibilities.

Original	HookRelay
`conduit-engine-main`	`hookrelay`
`verifyWebhook.ts`	`verify-webhook.ts`
`cdtsig_sha256=` signature prefix	`hrelay_sha256=`
`X-Conduit-*` headers	`X-HookRelay-*`
`cdt_` API key prefix	`hrelay_`

Original	HookRelay
`conduit-*` Docker containers	`hookrelay-*`
Duplicated retry logic	Shared `scheduleRetry` helper
Repeated inline `resolveUserId` logic	Shared helper function

These changes establish consistent project naming and reduce duplicated logic.

Roadmap

- [ ] Add status filtering to delivery logs.
- [ ] Build a frontend dashboard.
- [ ] Add unit tests for signature verification, retry scheduling, and delivery handling.
- [ ] Add integration tests for the inbound-to-outbound pipeline.
- [ ] Improve delivery filtering and operational visibility.
- [ ] Document the complete request and response schemas.
- [ ] Strengthen production deployment and security guidance.

Contributing

Contributions, bug reports, and feature requests are welcome.

1. Open an issue before making significant changes.
2. Fork the repository.
3. Create a branch for your changes.
4. Follow the existing TypeScript conventions.
5. Test your changes.
6. Submit a pull request describing the change and its rationale.

License

HookRelay is open-source software licensed under the MIT License.

Author

Built by Reabot6 at **SFG**.

HookRelay — receive once, deliver reliably.